UTT CONNECT · DOCUMENTACIÓN TÉCNICA

Cómo leer la base de datos

Módulo de autenticación, autorización y auditoría

Guía introductoria para interpretar el esquema PostgreSQL: tablas, relaciones, restricciones, índices, disparadores, particiones y el flujo de autenticación con enlace mágico.

MOTOR DE BASE DE DATOS

PostgreSQL 15+

PROYECTO

UTT Connect

EQUIPO

Equipo 1

FECHA

Octubre de 2026

NAVEGACIÓN

Contenido

Introducción

1. Cómo leer una definición SQL

2. Mapa mental de las tablas

3. Catálogos iniciales

4. Tabla central: users

5. Identidad externa: user_identities

6. Control de acceso: RBAC

7. El flujo de autenticación: magic_link_requests

8. Auditoría: audit_event_types y audit_events

9. El trigger de updated_at

10. Relaciones explicadas en lenguaje normal

11. Flujo completo de inicio de sesión

12. Consultas para practicar

13. Decisiones de diseño que vale la pena aprender

14. Cosas que el esquema no hace

15. Checklist para leer cualquier tabla nueva

Resumen final

CÓMO ESTÁ ORGANIZADA ESTA GUÍA

Secciones 1–2: conceptos básicos y mapa general del esquema. **Secciones 3–9:** explicación tabla por tabla. **Secciones 10–12:** relaciones, flujo de autenticación y consultas de práctica. **Secciones 13–15:** criterios de diseño, límites del esquema y método para leer cualquier tabla nueva.

Introducción

Este script **no crea toda la base de datos de UTT Connect**. Crea únicamente el módulo que responde cuatro preguntas:

1. **¿Quiénes son los usuarios?** → `users`
2. **¿Cómo demuestran su identidad?** → `user_identities`, `magic_link_requests`
3. **¿Qué pueden hacer?** → `roles`, `permissions`, `user_roles`, `role_permissions`
4. **¿Qué pasó y cuándo?** → `audit_events`

Las tablas de avisos, departamentos, chat y adjuntos no están aquí: pertenecen a otros módulos.

> **Cómo usar esta guía.** No es necesario memorizar todo. Primero lee los conceptos de la sección 1, después el mapa de la sección 2 y finalmente cada tabla. Al terminar, la sección 15 ofrece una lista de preguntas que puedes aplicar a cualquier tabla nueva.

1. Cómo leer una definición SQL

Antes de ver las tablas, aprende a "descomponer" una línea. Por ejemplo:

```
status_code varchar(32) NOT NULL DEFAULT 'pending'
    REFERENCES user_statuses(code)
```

Se lee así:

Parte	Significado
`status_code`	Nombre de la columna
`varchar(32)`	Tipo de dato: texto de máximo 32 caracteres
`NOT NULL`	Obligatoria: no puede quedar vacía
`DEFAULT 'pending'`	Si no se envía valor, se guarda `'pending'`
`REFERENCES user_statuses(code)`	Llave foránea: el valor debe existir en `user_statuses.code`

Otras piezas que aparecerán:

Pieza	Qué significa
`PRIMARY KEY`	Identifica de forma única cada fila
`FOREIGN KEY` / `REFERENCES`	Apunta a una fila de otra tabla
`UNIQUE`	No permite valores repetidos
`CHECK (...)`	Regla de negocio que la base de datos valida
`CREATE INDEX`	Acelera búsquedas; no cambia la lógica
`TRIGGER`	Código que se ejecuta automáticamente ante un evento
`BEGIN` / `COMMIT`	Todo el script se aplica como una sola unidad: si algo falla, no se guarda nada
`PARTITION BY RANGE`	Divide una tabla grande en pedazos por rango de fechas
`ON DELETE CASCADE`	Si se borra el padre, se borran sus hijos
`ON DELETE RESTRICT`	Impide borrar el padre si tiene hijos
`ON DELETE SET NULL`	Si se borra el padre, la referencia queda vacía

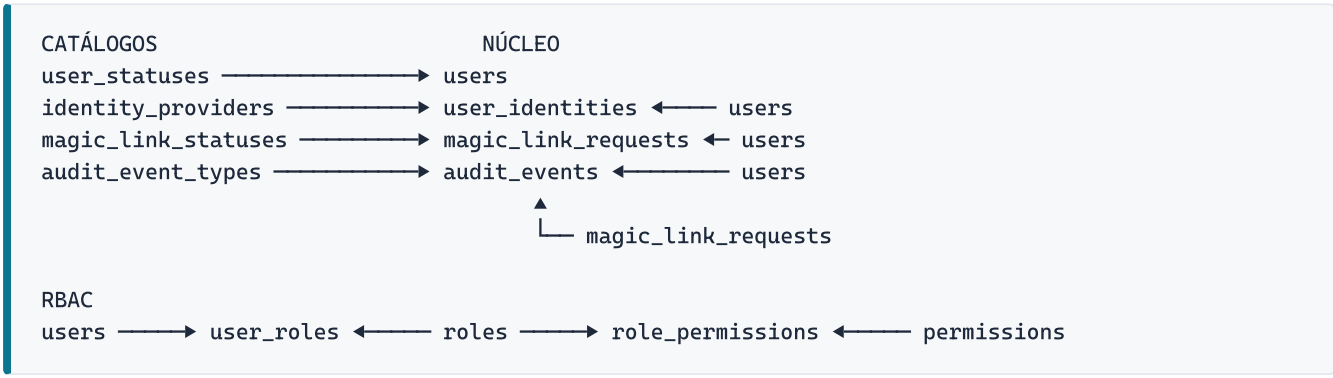
Tipos de datos que usa el script

Tipo	Para qué sirve	Ejemplo
`uuid`	Identificador único larguísimo	`gen_random_uuid()`

Tipo	Para qué sirve	Ejemplo
`varchar(n)`	Texto con límite	`varchar(160)`
`text`	Texto sin límite práctico	`continue_url`
`citext`	Texto que **no distingue mayúsculas/minúsculas**	Correos
`timestamptz`	Fecha y hora con zona horaria	`created_at`
`boolean`	Verdadero/falso	`succeeded`
`smallint`	Entero pequeño	`provider_id`
`jsonb`	Documento JSON binario, consultable	`payload`, `claims_snapshot`
`bytea`	Datos binarios	`client_nonce_hash`
`inet`	Dirección IP válida	`requested_ip`

> **Detalle importante:** `citext` hace que `Juan@UTT.edu.mx` y `juan@utt.edu.mx` se consideren el mismo correo. Por eso se usa en `users.email` y en los correos de los enlaces mágicos.

2. Mapa mental de las tablas



Hay dos tipos de tablas:

1. **Tablas de catálogo:** listas cerradas de valores válidos. Evitan errores de escritura y permiten cambiar descripciones sin tocar el código.
2. **Tablas de hechos o relaciones:** registran lo que ocurre: usuarios, identidades, asignaciones, enlaces y eventos.

3. Catálogos iniciales

`user_statuses`

Guarda los estados posibles de un usuario.

```
code         varchar(32) PRIMARY KEY
description  varchar(255) NOT NULL
```

Se insertan cuatro estados:

Código	Significado
`pending`	Se registró, pero aún no completa la autenticación
`active`	Puede usar el sistema
`suspended`	Bloqueado temporalmente
`disabled`	Deshabilitado

Por qué una tabla y no un simple texto: si `users.status_code` fuera texto libre, alguien podría escribir `activo`, `active` o `ACTIVO`. Con la llave foránea, la base de datos solo acepta los códigos definidos.

Lo que la base de datos no controla: que un usuario pase de `pending` a `active` en el orden correcto. Eso lo decide la aplicación.

`identity_providers`

Guarda los proveedores externos de identidad. Solo se inserta uno:

Código	Descripción
`firebase`	Firebase Authentication

Se usa `smallint GENERATED ALWAYS AS IDENTITY` como llave primaria: PostgreSQL genera automáticamente 1, 2, 3… y no se permite insertar el número manualmente.

Por qué existe: hoy la identidad la valida Firebase, pero mañana podría añadirse Google, Microsoft o un proveedor institucional. La tabla evita modificar el resto del modelo.

`magic_link_statuses`

Estados por los que pasa una solicitud de enlace mágico:

Código	Significado	¿Terminal?
`requested`	Solicitud recibida	No

Código	Significado	¿Terminal?
`sent`	Enlace enviado por Firebase	No
`consumed`	Enlace usado correctamente	Sí
`expired`	Enlace expirado	Sí
`invalid`	Enlace inválido	Sí
`failed`	Falló el envío o la validación	Sí
`revoked`	Solicitud revocada	Sí

La columna `is_terminal` le dice a la aplicación si el estado ya no debería cambiar. La base de datos no impide volver de `consumed` a `sent`; esa regla vive en el backend.

`audit_event_types`

Catálogo de los diez tipos de evento que se pueden auditar:

Código	Significado
`auth.magic_link_requested`	Se solicitó un enlace
`auth.magic_link_sent`	Se envió el enlace
`auth.login_succeeded`	Inicio de sesión correcto
`auth.login_failed`	Inicio de sesión fallido
`auth.link_expired`	El enlace expiró
`auth.link_invalid`	El enlace fue inválido
`user.created`	Usuario creado
`user.updated`	Usuario actualizado
`role.granted`	Rol concedido
`role.revoked`	Rol revocado

La convención `dominio.acción` facilita filtrar, por ejemplo, todos los eventos que empiezan con `auth.`.

4. Tabla central: `users`

Representa a una persona dentro del sistema.

Columna	Tipo	Significado
`id`	`uuid`	Identificador interno
`email`	`citext`	Correo institucional, único
`display_name`	`varchar(160)`	Nombre visible
`status_code`	`varchar(32)`	Estado actual; apunta a `user_statuses`
`email_verified_at`	`timestamptz`	Cuándo se verificó el correo
`last_login_at`	`timestamptz`	Último acceso
`created_at`	`timestamptz`	Fecha de creación
`updated_at`	`timestamptz`	Última modificación
`deleted_at`	`timestamptz`	Borrado lógico

Reglas importantes

```
CONSTRAINT users_email_not_blank CHECK (btrim(email::text) <> '')
```

Obliga a que el correo no sea solo espacios. `btrim` elimina espacios al inicio y al final antes de comparar.

```
CONSTRAINT users_verified_email_consistency CHECK (
    email_verified_at IS NULL OR status_code <> 'pending'
)
```

Si el correo ya fue verificado, el usuario **no puede seguir en estado** `pending`. Es decir, la aplicación debe actualizar ambas cosas en la misma operación: poner `email_verified_at` y cambiar el estado a `active`.

Borrado lógico

`deleted_at` no borra la fila. Solo marca la fecha. La aplicación debe filtrar:

```
WHERE deleted_at IS NULL
```

Ventaja: conserva historial y relaciones.

Consecuencia: el correo sigue ocupando el `UNIQUE`; no se puede reutilizar el mismo correo para una cuenta nueva mientras la fila exista. Si se quisiera permitir, habría que cambiar el índice único por uno parcial:

```
CREATE UNIQUE INDEX ... ON users (email) WHERE deleted_at IS NULL;
```

Índices de `users`

```
CREATE INDEX idx_users_status_active
    ON users (status_code, created_at DESC)
    WHERE deleted_at IS NULL;
```

Es un **índice parcial**: solo indexa usuarios no borrados. Sirve para listar, por ejemplo, usuarios activos ordenados por antigüedad.

```
CREATE INDEX idx_users_last_login
    ON users (last_login_at DESC)
    WHERE deleted_at IS NULL;
```

Acelera reportes de "últimos accesos".

Lo que no está en `users`

- **No hay contraseña.** La autenticación se delega a Firebase mediante enlace mágico. Por eso no existe `password_hash`.
- **No se valida el dominio `@utt.edu.mx` en la base de datos.** Solo se valida que el correo no esté vacío. La restricción del dominio debe hacerla el backend o, si se quisiera reforzar aquí, un `CHECK` con expresión regular.
- **No hay trigger que impida cambiar el correo.** El requisito R12 del README menciona un trigger de inmutabilidad, pero este script no lo incluye; hoy la inmutabilidad depende de la aplicación.

5. Identidad externa: `user_identities`

Relaciona un usuario local con una cuenta de un proveedor externo.

Columna	Significado
`id`	Identificador del vínculo
`user_id`	Usuario local
`provider_id`	Proveedor (Firebase)
`provider_subject`	Identificador del usuario **en el proveedor** (UID de Firebase)
`provider_email`	Correo que reporta el proveedor
`claims_snapshot`	Copia JSON de los datos del token
`linked_at`	Cuándo se vinculó
`last_verified_at`	Última vez que se validó

Restricciones clave

```
UNIQUE (provider_id, provider_subject)
```

Un mismo UID de Firebase no puede estar vinculado a dos usuarios locales.

```
UNIQUE (user_id, provider_id)
```

Un usuario local solo puede tener **una identidad por proveedor**. Puede tener varias identidades si son de proveedores distintos.

```
ON DELETE CASCADE
```

Si se borra físicamente el usuario, sus identidades también se borran. Como normalmente se usa borrado lógico, esto casi nunca se dispara.

Por qué `claims_snapshot`: el token de Firebase puede incluir nombre, correo, foto, etc. Guardar una copia permite auditar qué datos reportó el proveedor en ese momento, sin depender de que Firebase los conserve igual.

Detalle menor: existe un índice `idx_user_identities_user (user_id)`, pero el `UNIQUE (user_id, provider_id)` ya crea un índice que empieza por `user_id`. Ese índice adicional es redundante para búsquedas por usuario, aunque no rompe nada; solo añade costo de escritura.

6. Control de acceso: RBAC

RBAC significa **Role-Based Access Control**: los permisos no se asignan directamente a las personas, sino a roles, y las personas reciben roles.

`roles`

Columna	Significado
`id`	Identificador
`code`	Código único, por ejemplo `admin`
`name`	Nombre legible
`description`	Descripción
`is_system`	Indica si es un rol protegido del sistema
`created_at`, `updated_at`	Fechas

La regla:

```
CHECK (code ~ '^[a-z][a-z0-9_.-]{1,63}$')
```

Obliga a que el código:

- empiece con letra minúscula,
- tenga entre 2 y 64 caracteres,
- solo use minúsculas, números, punto, guion y guion bajo.

Importante: el script no inserta los roles `lector`, `departamento` ni `admin`. Eso debe hacerlo la aplicación o una migración posterior. `is_system` tampoco impide borrar el rol por sí solo; es una marca para que la aplicación lo proteja.

`permissions`

Representa una acción concreta:

Columna	Significado
`id`	Identificador
`code`	Código único, por ejemplo `avisos:crear`
`description`	Qué permite
`created_at`	Fecha

La expresión regular permite códigos como:

- `usuarios:leer`
- `avisos:crear`
- `auditoria.consultar`

`role_permissions`

Tabla puente entre roles y permisos. Es una relación **muchos a muchos**: un rol tiene muchos permisos y un permiso puede estar en muchos roles.

```
PRIMARY KEY (role_id, permission_id)
```

La llave primaria compuesta impide repetir la misma combinación. `ON DELETE CASCADE` en ambas llaves foráneas significa que si se borra un rol o un permiso, sus asignaciones desaparecen.

El índice invertido:

```
CREATE INDEX idx_role_permissions_permission
    ON role_permissions (permission_id, role_id);
```

sirve para responder "¿qué roles tienen este permiso?".

`user_roles`

Asigna un rol a un usuario, con metadatos:

Columna	Significado
`user_id`	Usuario
`role_id`	Rol
`granted_by`	Quién lo concedió
`granted_at`	Cuándo
`expires_at`	Cuándo expira, si aplica

```
PRIMARY KEY (user_id, role_id)
```

Un usuario no puede tener dos veces el mismo rol.

```
granted_by uuid REFERENCES users(id) ON DELETE SET NULL
```

Es una **autorreferencia**: `users` apunta a sí misma. Si se borra físicamente al administrador que concedió el rol, la asignación se conserva y `granted_by` queda vacío.

```
CONSTRAINT user_role_expiration
    CHECK (expires_at IS NULL OR expires_at > granted_at)
```

Una asignación no puede nacer ya expirada.

Comportamientos al borrar:

Borras…	Qué pasa
Un usuario	Se borran sus asignaciones (`CASCADE`)
Un rol	No se puede borrar si está asignado (`RESTRICT`)
Al usuario que concedió	La asignación queda, pero `granted_by` se vuelve `NULL`

Detalle de diseño: una asignación expirada **no se borra automáticamente**. Sigue en la tabla. Las consultas deben filtrar:

```
WHERE expires_at IS NULL OR expires_at > now()
```

Además, como la llave primaria es `(user_id, role_id)`, volver a conceder el mismo rol después de que expire requiere actualizar `expires_at` o borrar la fila anterior. Si se quisiera conservar historial completo de concesiones, habría que diseñar una tabla de historial aparte.

7. El flujo de autenticación: `magic_link_requests`

Un **magic link** es un enlace de un solo uso que se envía al correo. Al abrirlo, el usuario demuestra que controla ese correo.

Columnas

Columna	Significado
`id`	Identificador de la solicitud
`user_id`	Usuario, si ya existe; puede ser `NULL`
`email`	Correo al que se envía
`status_code`	Estado actual
`correlation_id`	Identificador único para rastrear todo el flujo
`continue_url`	A dónde volver después de autenticarse
`client_nonce_hash`	Hash de un valor secreto del navegador
`requested_ip`	IP que solicitó el enlace
`user_agent`	Navegador o cliente
`firebase_event_id`	Identificador del evento en Firebase
`requested_at`	Cuándo se solicitó
`sent_at`	Cuándo se envió
`expires_at`	Cuándo caduca
`consumed_at`	Cuándo se usó
`failure_code`	Código de fallo
`failure_detail`	Detalle del fallo
`metadata`	Datos extra en JSON

¿Por qué `user_id` puede ser `NULL`?

Porque alguien puede solicitar un enlace **antes de existir** en `users`. En el primer acceso se crea el usuario. Por eso la relación es opcional y usa `ON DELETE SET NULL`: si el usuario se borra, la solicitud histórica se conserva.

¿Por qué `correlation_id`?

Permite seguir una misma autenticación a través de logs, solicitudes y eventos de auditoría. Es único y se genera automáticamente.

¿Por qué `client_nonce_hash`?

Para vincular la solicitud al navegador que la inició sin guardar el secreto en texto plano. Se guarda el **hash**, no el valor original. Es una buena práctica de seguridad.

Reglas de consistencia

```
CHECK (expires_at > requested_at)
```

El enlace no puede caducar antes de haberse solicitado.

```
CHECK (sent_at IS NULL OR sent_at >= requested_at)
```

No puede haberse enviado antes de solicitarse.

```
CHECK (consumed_at IS NULL OR consumed_at >= requested_at)
```

No puede haberse usado antes de solicitarse.

```
CHECK (
    (status_code = 'consumed' AND consumed_at IS NOT NULL)
    OR
    (status_code <> 'consumed' AND consumed_at IS NULL)
)
```

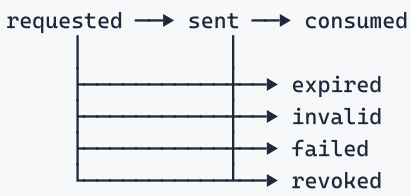
Esta es la regla más importante: **si el estado es `consumed`, debe existir `consumed_at`; y si no es `consumed`, `consumed_at` debe ser nulo**. Así se evita un estado "usado" sin fecha o una fecha de uso con estado "pendiente".

Índices

Índice	Para qué
`(email, requested_at DESC)`	Ver solicitudes recientes de un correo y aplicar límites
`(user_id, requested_at DESC)` parcial	Historial de un usuario
`(expires_at)` parcial para `requested` / `sent`	Encontrar enlaces pendientes de expirar
`(firebase_event_id)` parcial	Rastrear eventos de Firebase

Lo que no hay: una restricción que impida varias solicitudes pendientes para el mismo correo. Eso se controla con *rate limiting* en el backend, no con un `UNIQUE`.

Máquina de estados



La base de datos guarda el estado, pero **no obliga** a seguir ese orden. La lógica de transiciones está en NestJS.

8. Auditoría: `audit_event_types` y `audit_events`

`audit_events`

Es la bitácora del sistema. Cada fila responde: **quién hizo qué, sobre quién, cuándo, desde dónde y si funcionó**.

Columna	Significado
`occurred_at`	Momento del evento
`id`	Identificador del evento
`event_type_code`	Tipo de evento
`actor_user_id`	Quién ejecutó la acción
`subject_user_id`	Sobre quién se ejecutó
`magic_link_request_id`	Enlace relacionado, si aplica
`correlation_id`	Hilo de trazabilidad
`resource_type`	Tipo de recurso afectado
`resource_id`	Identificador del recurso
`source_ip`	IP de origen
`user_agent`	Cliente
`succeeded`	Si la acción tuvo éxito
`payload`	Datos adicionales en JSON

Diferencia entre actor y sujeto:

- En un login propio: `actor_user_id` y `subject_user_id` pueden ser el mismo usuario.
- En una concesión de rol: el actor es el administrador; el sujeto es el usuario que recibe el rol.

`resource_type` y `resource_id` son una referencia **polimórfica**: pueden apuntar a un rol, a un usuario, a un aviso, etc. No tienen llave foránea porque el tipo de recurso es variable. Es flexible, pero la base de datos no garantiza que el recurso exista.

Particionamiento

```
PRIMARY KEY (occurred_at, id)
) PARTITION BY RANGE (occurred_at);
```

La tabla está dividida por rangos de fecha. PostgreSQL exige que la llave primaria incluya la columna de partición; por eso es `(occurred_at, id)` y no solo `id`.

El script crea:

```
CREATE TABLE audit_events_default
    PARTITION OF audit_events DEFAULT;
```

Esa partición **atrapa todo** lo que no caiga en una partición específica. Es una red de seguridad, pero en producción conviene crear particiones mensuales, por ejemplo:

```
audit_events_2026_10
audit_events_2026_11
audit_events_2026_12
```

Ventajas de particionar:

- Consultar un mes solo lee esa partición.
- Borrar historia vieja es barato: se elimina la partición completa.
- La tabla principal no se vuelve inmanejable.

Cuidado operativo: si la partición por defecto acumula muchos datos, crear una nueva partición puede requerir revisar y mover esas filas. Hay que monitorearla.

Índices de auditoría

Índice	Consulta que acelera
`(actor_user_id, occurred_at DESC)`	"¿Qué hizo este usuario?"
`(subject_user_id, occurred_at DESC)`	"¿Qué le pasó a este usuario?"
`(event_type_code, occurred_at DESC)`	"Todos los logins fallidos del mes"
`(correlation_id, occurred_at DESC)`	"Sigue este flujo completo"
`(resource_type, resource_id, occurred_at DESC)`	"Historial de este recurso"

Lo que la auditoría no garantiza por sí sola

- **No es inmutable:** no hay trigger que impida `UPDATE` o `DELETE`. En producción se deben revocar esos privilegios al usuario de la aplicación.
- **Las llaves foráneas usan `ON DELETE SET NULL`:** si se borra físicamente un usuario, el evento se conserva, pero se pierde el vínculo. Por eso el `payload` debería guardar una copia de los datos relevantes (correo, rol, etc.).
- **`payload` es libre:** su calidad depende de lo que escriba la aplicación. La única validación es que sea un objeto JSON.

9. El trigger de `updated_at`

```
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN
    NEW.updated_at := now();
    RETURN NEW;
END;
$$;
```

`NEW` representa la fila **después** del cambio. El trigger la modifica y la devuelve para que se guarde.

Se aplica a:

```
CREATE TRIGGER trg_users_set_updated_at
BEFORE UPDATE ON users
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_roles_set_updated_at
BEFORE UPDATE ON roles
FOR EACH ROW
EXECUTE FUNCTION set_updated_at();
```

Por qué solo en `users` y `roles`: son las únicas tablas con columna `updated_at`. `permissions` solo tiene `created_at`; los enlaces y la auditoría son registros históricos que no deberían "actualizarse".

Detalle técnico: `now()` devuelve la hora de inicio de la transacción, no la del instante exacto de cada fila. Si en una misma transacción actualizas diez filas, las diez tendrán el mismo `updated_at`. Para la mayoría de los casos es correcto; si se necesitara precisión por fila, se usaría `clock_timestamp()`.

10. Relaciones explicadas en lenguaje normal

Relación	Lectura
`user_statuses` → `users`	Un estado puede tener muchos usuarios; un usuario tiene un estado
`identity_providers` → `user_identities`	Un proveedor puede respaldar muchas identidades; una identidad pertenece a un proveedor
`users` → `user_identities`	Un usuario puede tener varias identidades (una por proveedor)
`users` ↔ `roles` mediante `user_roles`	Muchos a muchos, con fecha, quién concedió y expiración
`roles` ↔ `permissions` mediante `role_permissions`	Muchos a muchos
`magic_link_statuses` → `magic_link_requests`	Un estado clasifica muchas solicitudes
`users` → `magic_link_requests`	Un usuario puede solicitar muchos enlaces; el enlace puede no tener usuario todavía
`audit_event_types` → `audit_events`	Un tipo clasifica muchos eventos
`users` → `audit_events`	Un usuario puede ser actor o sujeto de muchos eventos
`magic_link_requests` → `audit_events`	Un enlace puede generar varios eventos

11. Flujo completo de inicio de sesión

Así se usan las tablas anteriores en orden:

1. **El usuario escribe su correo** en Angular.
2. **NestJS inserta** en `magic_link_requests`: `email`, `continue_url`, `expires_at` (por ejemplo, `now() + interval '15 minutes'`), `requested_ip`, `user_agent`, `client_nonce_hash` y `status_code = 'requested'`.
3. **Firebase envía el correo.** El backend actualiza `status_code = 'sent'`, `sent_at = now()` y `firebase_event_id`.
4. **El usuario abre el enlace.** Firebase valida y devuelve un token.
5. **NestJS valida el token** y obtiene el UID y el correo.
6. **En una transacción**, el backend: - busca al usuario por correo; - si no existe, lo crea en `users`; - crea o actualiza `user_identities` con `(provider_id, provider_subject)`; - actualiza `email_verified_at` y cambia el estado a `active` **a la vez**, porque el `CHECK` lo exige; - actualiza `last_login_at`; - marca el enlace como `consumed` y llena `consumed_at`.
7. **Resuelve roles y permisos** consultando `user_roles`, `roles`, `role_permissions` y `permissions`.
8. **Registra la auditoría**: `auth.magic_link_requested`, `auth.magic_link_sent`, `auth.login_succeeded`, `user.created`, etc.
9. **Devuelve la sesión** y Angular muestra el dashboard según el rol.

Conceder un rol

```
INSERT INTO user_roles (user_id, role_id, granted_by, expires_at)
VALUES ($1, $2, $3, $4);
```

Luego se inserta un evento `role.granted` en `audit_events`.

Expirar enlaces

Un proceso programado puede ejecutar:

```
UPDATE magic_link_requests
SET status_code = 'expired'
WHERE status_code IN ('requested', 'sent')
  AND expires_at < now();
```

Y registrar `auth.link_expired` en la auditoría.

12. Consultas para practicar

Usuarios activos

```
SELECT id, email, display_name, last_login_at
FROM users
WHERE deleted_at IS NULL
  AND status_code = 'active'
ORDER BY last_login_at DESC NULLS LAST;
```

`NULLS LAST` coloca al final a quienes nunca han entrado.

Roles vigentes de un usuario

```
SELECT r.code, r.name, ur.granted_at, ur.expires_at
FROM user_roles ur
JOIN roles r ON r.id = ur.role_id
WHERE ur.user_id = $1
  AND (ur.expires_at IS NULL OR ur.expires_at > now());
```

`$1` es un parámetro. Usar parámetros evita inyección SQL.

Permisos efectivos

```
SELECT DISTINCT p.code
FROM user_roles ur
JOIN role_permissions rp ON rp.role_id = ur.role_id
JOIN permissions p ON p.id = rp.permission_id
WHERE ur.user_id = $1
  AND (ur.expires_at IS NULL OR ur.expires_at > now());
```

Enlaces pendientes de expirar

```
SELECT id, email, requested_at, expires_at
FROM magic_link_requests
WHERE status_code IN ('requested', 'sent')
  AND expires_at < now();
```

Últimos eventos de un usuario

```
SELECT occurred_at, event_type_code, succeeded, source_ip, payload
FROM audit_events
WHERE subject_user_id = $1
ORDER BY occurred_at DESC
LIMIT 50;
```

Trazabilidad de una autenticación

```
SELECT occurred_at, event_type_code, actor_user_id, subject_user_id, succeeded, payload
FROM audit_events
WHERE correlation_id = $1
ORDER BY occurred_at;
```

13. Decisiones de diseño que vale la pena aprender

Decisión	Beneficio	Costo o cuidado
Tablas de catálogo en vez de texto libre	Evita errores y centraliza descripciones	Más joins
`uuid` como llave	Se puede generar sin consultar la base; útil en sistemas distribuidos	Ocupa más espacio que un entero
`timestamptz`	Guarda en UTC y respeta zona horaria	Hay que formatear en la interfaz
`citext` en correos	Comparación sin distinguir mayúsculas	Requiere la extensión
Borrado lógico	Conserva historial	Todas las consultas deben filtrar `deleted_at`
`jsonb` para `payload` y `claims`	Flexibilidad para datos variables	No hay validación de esquema interna
Índices parciales	Indexan solo lo relevante; ocupan menos	Hay que diseñarlos según las consultas reales
Particionamiento por fecha	Rendimiento y borrado barato de historia	Requiere mantenimiento de particiones
`ON DELETE CASCADE`	Limpieza automática de hijos	Puede borrar más de lo esperado
`ON DELETE RESTRICT`	Protege datos en uso	Obliga a limpiar antes de borrar
`ON DELETE SET NULL`	Conserva el hecho histórico	Pierde el vínculo
Delegar identidad a Firebase	No se almacenan contraseñas	Dependencia de un servicio externo
RBAC con roles y permisos	Los permisos se administran sin tocar código	Más tablas y consultas
Auditoría separada del negocio	No contamina las tablas operativas	Puede crecer muy rápido

14. Cosas que el esquema no hace

Es importante saberlo para no asumir garantías que no existen:

1. **No valida el dominio `@utt.edu.mx`.** Solo valida que el correo no esté vacío.
2. **No impide cambiar el correo.** El requisito de inmutabilidad debe implementarse con un trigger o en la aplicación.
3. **No obliga a que un usuario `active` tenga correo verificado.** Solo impide que un usuario verificado siga en `pending`.
4. **No borra roles expirados.** Las consultas deben filtrar por `expires_at`.
5. **No impide modificar o borrar la auditoría.** Hay que restringir privilegios.
6. **No crea particiones mensuales.** Solo crea la partición por defecto.
7. **No incluye los roles ni permisos iniciales.** Hay que insertarlos.
8. **No modela sesiones concurrentes.** El límite de dos sesiones del README se maneja en otra capa.
9. **No incluye contraseñas.** Este módulo usa magic link + Firebase, no OTP con contraseña.
10. **No incluye avisos, departamentos, chat ni archivos.** Es solo el módulo de identidad y auditoría.
11. **No es idempotente.** Volver a ejecutar el script falla porque las tablas ya existen. En producción se usan migraciones versionadas.
12. **No define permisos de usuario de base de datos.** El script asume que lo ejecuta un usuario con privilegios suficientes.

15. Checklist para leer cualquier tabla nueva

Cuando veas una tabla que no conoces, pregúntate en este orden:

1. **¿Qué nombre tiene?** → ¿Qué concepto del mundo real representa?
2. **¿Cuál es su llave primaria?** → ¿Cómo se identifica una fila?
3. **¿Qué llaves foráneas tiene?** → ¿Con qué se relaciona?
4. **¿Qué columnas son `NOT NULL`?** → ¿Qué datos son obligatorios?
5. **¿Qué `DEFAULT` tiene?** → ¿Qué se llena solo?
6. **¿Qué `UNIQUE` tiene?** → ¿Qué no puede repetirse?
7. **¿Qué `CHECK` tiene?** → ¿Qué reglas de negocio valida la base?
8. **¿Qué índices tiene?** → ¿Qué consultas se esperan?
9. **¿Tiene triggers?** → ¿Qué ocurre "invisiblemente" al insertar o actualizar?
10. **¿Qué pasa al borrar?** → ¿`CASCADE`, `RESTRICT` o `SET NULL`?
11. **¿Tiene datos iniciales?** → ¿Qué catálogos se sembraron?
12. **¿Qué no está validado aquí?** → ¿Qué queda en la aplicación?

Resumen final

La base de datos está organizada como un sistema de identidad moderno:

- `users` es la persona.
- `user_identities` dice qué cuenta externa la respalda.
- `magic_link_requests` registra el intento de autenticación.
- `roles` + `permissions` + **tablas puente** definen qué puede hacer.
- `audit_events` conserva la historia de lo ocurrido.
- Los **catálogos** (`user_statuses`, `identity_providers`, `magic_link_statuses`, `audit_event_types`) mantienen los valores válidos bajo control.
- Las **restricciones** (`CHECK`, `UNIQUE`, `FOREIGN KEY`) impiden datos incoherentes.
- Los **índices** hacen rápidas las consultas frecuentes.
- El **trigger** actualiza `updated_at` automáticamente.
- El **particionamiento** prepara la auditoría para crecer.